

UYGULAMALI REKÜRSİF ANALİZ

Çalışma Kağıdı | Akra-Bazzi İspatı · Subset Sum · Fibonacci

Bu kağıt üç konuyu bir araya getirir: (1) Akra-Bazzi çözümünün integral türetmesi, (2) Subset Sum problemi ve rekürsif ağaç analizi, (3) Fibonacci dizisinin rekürsif analizi ve sınır ispat tekniği.

1 1 Akra-Bazzi – İntegral Çözümünün Türetilmesi

Master Teorem'in genelleştirilmesi olan Akra-Bazzi, farklı boyutlu alt problemleri ele alır. Burada çözümün integral formülüne nasıl ulaşıldığını adım adım göreceğiz.

Bağıntı:

$$T(n) = \sum_{i=1}^k a_i T\left(\frac{n}{b_i}\right) + f(n)$$

p değeri: $\sum_{i=1}^k \frac{a_i}{b_i^p} = 1$ denkleminin çözümü.

1.1 – İki Terimli Durum: $T(n) = T(n/2) + T(n/3) + f(n)$

p denklemini yazıp sezgisel olarak çözelim; ardından integral formülünü açıklayalım.

Örnek 1.1

$$T(n) = T(n/2) + T(n/3) + n$$

Adım 1 – p denklemini kur:

$$\frac{1}{2^p} + \frac{1}{3^p} = 1$$

$p = 0$: $\frac{1}{1} + \frac{1}{1} = 2 > 1$ $p = 1$: $\frac{1}{2} + \frac{1}{3} = \frac{5}{6} < 1$
 $\Rightarrow p \in (0, 1)$, sayısal çözüm: $p \approx 0.787$

Adım 2 – c ile p karşılaştır: $f(n) = n = n^1$, yani $c = 1 > p \approx 0.787$

Adım 3 – İntegral hesapla:

$$\int_1^n \frac{f(u)}{u^{p+1}} du = \int_1^n \frac{u}{u^{p+1}} du = \int_1^n u^{-p} du = \left[\frac{u^{1-p}}{1-p} \right]_1^n = \frac{n^{1-p} - 1}{1-p} \sim n^{1-p}$$

Adım 4 – Kapalı form:

$$T(n) = \Theta(n^p(1 + n^{1-p})) = \Theta(n^p + n) = \boxed{\Theta(n)}$$

1.2 – İntegral Hesabı: $f(n) = n^c$ Durumları

$f(n) = n^c$ olduğunda integralin nasıl davrandığını genelleştirelim.

$$I = \int_1^n \frac{u^c}{u^{p+1}} du = \int_1^n u^{c-p-1} du$$

Koşul	İntegral Sonucu	$T(n)$
$c < p$	$\frac{u^{c-p}}{c-p} \Big _1^n \rightarrow \text{sabit (yakınsak)}$	$\Theta(n^p)$
$c = p$	$\int_1^n u^{-1} du = \ln n$	$\Theta(n^p \log n)$
$c > p$	$\frac{n^{c-p} - 1}{c-p} \sim n^{c-p}$	$\Theta(n^p \cdot n^{c-p}) = \Theta(n^c)$

Pratik Soru 1

(a) $T(n) = T(n/2) + T(n/4) + n^2$ için p denklemini yazın ve $p < 1$ olduğunu gösterin. Ardından $T(n)$ karmaşıklığını bulun.

$\left(\frac{1}{2}\right)^p + \left(\frac{1}{4}\right)^p = 1$ $\left(\frac{1}{2}\right)^p \left(1 + \left(\frac{1}{2}\right)^p\right) = 1$ $1 + \left(\frac{1}{2}\right)^p = 2^p$ $\frac{1}{2^p} = 2^p - 1$ bu durum sadece $p < 1$ için sağlanır.

(b) $T(n) = T(n/3) + T(2n/3) + 1$ bağıntısı için p değerini bulun. $c = 0$ iken $T(n)$ nedir?

$\left(\frac{1}{3}\right)^p + \left(\frac{2}{3}\right)^p = 1 \Rightarrow p = 1$ $c < p$ $0 < 1$ $O(n^p) \Rightarrow O(n)$

$c > p$
 $O(n^2)$

2 1

Subset

Sum Problemi – Rekürsif Analiz

Subset Sum, bir kümeden elemanlar seçilerek hedef toplamın elde edilip edilemeyeceğini araştırır. Rekürsif çözüm her eleman için iki seçenek üretir: dahil et veya etme.

2.1 – Algoritma

Örnek 2.1

```
bool subsetSum(int set[], int n, int target) {
    if (target == 0) return true;          // hedef bulundu
    if (n == 0) return false;             // eleman kalmadı
    return subsetSum(set, n-1, target)     // n. eleman dahil edilmez
        || subsetSum(set, n-1, target - set[n-1]); // n. eleman dahil edilir
}
```

Yineleme bağıntısı (en kötü durum):

$$T(n) = 2T(n-1) + c, \quad T(0) = c$$

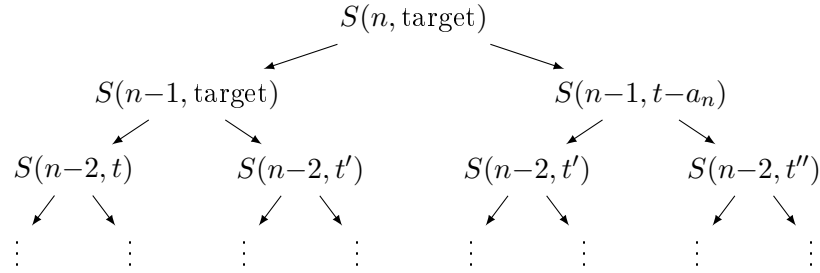
$T(n-1) = 2T(n-2) + c$
 $T(n) = 2^2 T(n-2) + c + 2c$
 $T(n) = 2^i T(n-i) + c(2^i - 1)$
 $n-i=0$
 $n=i$
 $T(n) = 2^n T(0) + c(2^n - 1)$
 $T(n) = c(2^{n+1} - 1) \Rightarrow O_n(2^n)$

Kapalı form: (önceki dersten)

$$T(n) = c(2^{n+1} - 1) = \boxed{\Theta(2^n)}$$

2.2 – Rekürsif Çağrı Ağacı

Her düğüm iki dal açar. n derinlikte 2^n yaprak oluşur.



Seviye k : 2^k düğüm | Toplam düğüm: $\sum_{k=0}^n 2^k = 2^{n+1} - 1$

2.3 – En İyi ve En Kötü Durum

Durum	Ne zaman?	Karmaşıklık
En iyi	<code>target==0</code> hemen sağlanır	$T(n) = O(1)$
Ortalama	Çözüm ortada bulunur	$T(n) = O(2^{n/2})$
En kötü	Çözüm yok, tüm ağaç taramır	$T(n) = O(2^n)$

Pratik Soru 2

(a) `set = {3, 34, 4, 12, 5, 2}`, `target = 9` için rekürsif ağacın kaç düğüm içerdiğini hesaplayın (en kötü durum). $n=6$ $\sum_{i=0}^6 2^i \Rightarrow 2^{6+1} - 1 = 127$

(b) Subset Sum problemi neden **NP-complete**'tir? Polinom zamanlı çözüm biliniyor mu? $\hookrightarrow$

(c) `target==0` kontrolü neden `n==0` kontrolünden önce yazılmıştır? _____

3 1

Fibonacci

– Rekürsif Analiz ve Sınır İspatı

Fibonacci dizisi iki rekürsif çağrı içerdiğinden üstel zaman alır. Ancak tam karmaşıklık 2^n 'den biraz daha küçüktür – bunu geometrik seri ve sıkıştırma (squeeze) ile göstereceğiz.

3.1 – Temel Tanım

Örnek 3.1 – Fibonacci bağıntısı

$$T(n) = 2T(n-1) + c, \quad T(0) = c$$

Önceki dersten: $T(n) = c(2^{n+1} - 1)$

Ancak Fibonacci'nin gerçek yineleme bağıntısı:

$$F(n) = F(n-1) + F(n-2), \quad F(0) = 0, \quad F(1) = 1$$

$$T_F(n) = T_F(n-1) + T_F(n-2) + c, \quad T_F(0) = c, \quad T_F(1) = c$$

3.2 – Geometrik Seri ile Kapalı Form

$T(n) = 2T(n-1) + c$ bağıntısını açalım. Her adımda 2^k ile çarpılan c terimleri birikir.

Örnek 3.2 – Unrolling

$$T(n) = 2T(n-1) + c$$

$$T(n-1) = 2T(n-2) + c \Rightarrow T(n) = 2^2T(n-2) + 2c + c$$

$$T(n-2) = 2T(n-3) + c \Rightarrow T(n) = 2^3T(n-3) + 2^2c + 2c + c$$

$$\vdots$$

$$T(n) = 2^iT(n-i) + c \underbrace{(2^{i-1} + 2^{i-2} + \dots + 1)}_{= 2^i - 1}$$

$$n - i = 0 \Rightarrow i = n:$$

$$T(n) = 2^n \cdot T(0) + c(2^n - 1) = c \cdot 2^n + c(2^n - 1) = c(2^{n+1} - 1)$$

$$T(n) = c(2^{n+1} - 1) = \Theta(2^n)$$

3.3 – Sıkıştırma (Squeeze) ile Fibonacci Sınırı

Fibonacci'nin her çağrısı $T(n-1)$ ve $T(n-2)$ kullanır. İki tane $T(n-1)$ yerine bir $T(n-1)$ ve bir $T(n-2)$ olduğundan sonuç 2^n 'den küçüktür.

Fibonacci Rekürsif Çağrı Analizi:

P_n : n girişli Fibonacci çağrısının ürettiği *yaprak sayısının* 2^n 'e oranı.

$0 \leq P_n \leq 1$ olduğu gösterilebilir.

Kapalı form (sıkıştırma sınırları):

$$T(n) = (2^{n+1} - 2) P_n + \frac{(2^{n+1} - 2)(2^{n+1} - 1)}{2} \cdot \frac{1 - P_n}{n}$$

Sınır:

$$\frac{2^{n+1} - 2}{2} \leq T(n) \leq 2^{n+1} - 2$$

$$T(n) \in O(2^n)$$

Not: Altın oran $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ ile daha kesin sonuç:

$$F(n) = \frac{\phi^n - \psi^n}{\sqrt{5}}, \quad \psi = \frac{1 - \sqrt{5}}{2} \approx -0.618 \quad \Rightarrow \quad F(n) = \Theta(\phi^n) \approx \Theta(1.618^n)$$

3.4 – Naif vs. Memoization vs. Döngüsel

Yöntem	Zaman	Alan	Açıklama
Naif rekürsif	$\Theta(\phi^n)$	$\Theta(n)$ (yığıt)	Aynı alt problem defalarca
Memoization	$\Theta(n)$	$\Theta(n)$	Her alt problem 1 kez hesaplanır
Dinamik prog.	$\Theta(n)$	$\Theta(n)$	Alt-üst (bottom-up)
Alan opt. DP	$\Theta(n)$	$\Theta(1)$	Sadece son iki değer tutulur
Matris üstel.	$\Theta(\log n)$	$\Theta(1)$	$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

Pratik Soru 3

(a) Naif rekürsif Fibonacci $n = 6$ için kaç çağrı yapar? Ağacı çizerek sayın ve $2^6 = 64$ ile karşılaştırın. _____

(b) Memoization neden rekürsif maliyeti $\Theta(\phi^n)$ 'dan $\Theta(n)$ 'e düşürür? _____

(c) $F(10) = ?$ Binet formülünü kullanarak hesaplayın.

$$F(10) = \frac{1.618^{10} - (-0.618)^{10}}{\sqrt{5}} \approx \underline{\hspace{2cm}}$$

4 1 ve Genel Karşılaştırma

Özet

Problem / Bağıntı	Yöntem	$T(n)$	Bölüm
$T(n/2) + T(n/3) + n$	Akra-Bazzi	$\Theta(n)$	1.1
$T(n/2) + T(n/4) + n^2$	Akra-Bazzi	$\Theta(n^2)$	Pratik 1a
Subset Sum (en kötü)	Unrolling	$\Theta(2^n)$	2
$T(n) = 2T(n-1) + c$	Geometrik seri	$\Theta(2^n)$	3.2
Fibonacci (naif)	Sıkıştırma	$\Theta(\phi^n)$	3.3
Fibonacci (memo.)	Dinamik prog.	$\Theta(n)$	3.4

Ek Karma Sorular

S1. $T(n) = T(n/2) + T(n/3) + n^0$ için Akra-Bazzi uygulayın. _____

S2. Tower of Hanoi: $T(n) = 2T(n-1) + 1$, $T(1) = 1$. Kapalı formu bulun ve Subset Sum ile karşılaştırın. _____

S3. Subset Sum'ı dinamik programlama ile $O(n \cdot \text{target})$ 'a düşürmek mümkün. Bu hangi koşulda $O(2^n)$ 'den daha iyidir? _____

S4. Fibonacci için $\phi = (1 + \sqrt{5})/2$ neden çıkıyor? $x^2 = x + 1$ karakteristik denklemini çözün. _____